

UNIVERSIDAD SERGIO ARBOLEDA

**TALLER: DISEÑO E IMPLEMENTACIÓN DE UN
ALGORITMO GENÉTICO**

Introducción a la Inteligencia Artificial

Camilo Andrés Díaz García
Mario Eliecer Jiménez López
Juan David Andrade Gómez

*Universidad Sergio Arboleda
Facultad de Sistemas e Ingeniería
Bogotá D.C., 2026*

Índice

1. Punto 1 – Formulación del problema	2
2. Punto 2 – Población y función de aptitud	2
3. Punto 3 – Operadores genéticos	3
4. Punto 4 – Algoritmo genético completo	4
4.1. Punto 4 – Resultados de la ejecución completa	5
5. Punto 5 – Experimentación y análisis	5
5.1. Experimentos A, B y C	5
5.2. Gráficas de evolución	5
5.3. Explicación de la mejor solución encontrada	7
5.4. Conclusiones sobre la influencia de los parámetros	8

1 Punto 1 – Formulación del problema

El problema consiste en seleccionar el subconjunto de diez proyectos de innovación que maximiza el beneficio total sin superar un presupuesto de 50 unidades monetarias. Corresponde a una variante del problema de la mochila 0/1.

- a) **Variable de decisión.** Para cada proyecto $i = 1, \dots, 10$ se define $x_i \in \{0, 1\}$, donde $x_i = 1$ si el proyecto i se selecciona y $x_i = 0$ en caso contrario.
- b) **Función objetivo.** Maximizar el beneficio total:

$$B(X) = \sum_{i=1}^{10} b_i x_i$$

- c) **Restricción principal.** El costo total no puede superar el presupuesto disponible:

$$C(X) = \sum_{i=1}^{10} c_i x_i \leq 50$$

- d) **Representación de un individuo.** Cada individuo se representa mediante un cromosoma binario de diez genes, $X = (x_1, \dots, x_{10})$, donde cada gen indica la inclusión o exclusión del proyecto correspondiente.
- e) **Tamaño del espacio de búsqueda.** Existen $2^{10} = 1024$ combinaciones posibles.

Cálculo manual. Para el individuo $X = (1, 0, 1, 1, 0, 0, 1, 0, 1, 0)$, que selecciona los proyectos P1, P3, P4, P7 y P9:

$$C(X) = 12 + 11 + 8 + 6 + 5 = 42$$

$$B(X) = 24 + 23 + 15 + 11 + 9 = 82$$

Como $C(X) = 42 \leq 50$, la solución es válida.

2 Punto 2 – Población y función de aptitud

La población inicial se genera con la función `generar_poblacion(tamano)`, que crea `tamano` cromosomas binarios aleatorios de 10 genes mediante `generar_individuo()`. La función de aptitud implementada en `calcular_aptitud()` es:

$$\text{fitness}(X) = \begin{cases} B(X), & \text{si } C(X) \leq 50, \\ B(X) - \lambda (C(X) - 50), & \text{si } C(X) > 50, \end{cases} \quad \lambda = 5.$$

Si λ es demasiado pequeño, la penalización deja de ser efectiva: individuos que exceden el presupuesto pueden conservar una aptitud similar o incluso mayor que soluciones válidas, por lo que el algoritmo tiende a converger hacia cromosomas inválidos. Si λ es demasiado grande, cualquier exceso de costo, incluso mínimo, resulta en una aptitud fuertemente negativa; esto equivale a un descarte abrupto de esas soluciones, lo que reduce la diversidad genética disponible para la recombinación y puede dificultar que el algoritmo escape de óptimos locales.

A continuación se muestra la población inicial generada con $N = 20$ (semilla fija `random.seed(42)`), tal como la produce `punto2_poblacion.py`, con cromosoma, proyectos seleccionados, costo, beneficio, aptitud y validez de cada individuo:

Cuadro 1: Población inicial y función de aptitud (Punto 2)

Ind.	Cromosoma	Proyectos	Costo	Beneficio	Aptitud	Válido
1	0010000010	P3, P9	16	32	32.0	Sí
2	0000001011	P7, P9, P10	24	45	45.0	Sí
3	0011100100	P3, P4, P5, P8	38	73	73.0	Sí
4	1011101010	P1, P3, P4, P5, P7, P9	51	98	93.0	No
5	1100001000	P1, P2, P7	25	48	48.0	Sí
6	1111011010	P1, P2, P3, P4, P6, P7, P9	63	123	58.0	No
7	0001110100	P4, P5, P6, P8	41	78	78.0	Sí
8	0111001011	P2, P3, P4, P7, P9, P10	50	96	96.0	Sí
9	1010011110	P1, P3, P6, P7, P8, P9	58	114	74.0	No
10	0100000101	P2, P8, P10	30	57	57.0	Sí
11	1110011011	P1, P2, P3, P6, P7, P9, P10	68	133	43.0	No
12	0101001001	P2, P4, P7, P10	34	64	64.0	Sí
13	0011001100	P3, P4, P7, P8	35	68	68.0	Sí
14	0001000101	P4, P8, P10	31	59	59.0	Sí
15	1001111100	P1, P4, P5, P6, P7, P8	59	113	68.0	No
16	0010000000	P3	11	23	23.0	Sí
17	0010011001	P3, P6, P7, P10	44	87	87.0	Sí
18	0110001111	P2, P3, P7, P8, P9, P10	52	100	90.0	No
19	1000110000	P1, P5, P6	35	68	68.0	Sí
20	1010110010	P1, P3, P5, P6, P9	51	100	95.0	No

3 Punto 3 – Operadores genéticos

Selección por torneo. La función `seleccionar_padre()` elige tres individuos al azar de la población (tamaño de torneo 3) y retorna el de mayor aptitud entre ellos como padre.

Cruce de un punto. La función `cruzar()` escoge, con probabilidad $p_c = 0,80$, una posición de corte aleatoria entre 1 y 9, e intercambia los segmentos finales de ambos padres para producir dos hijos. Si no ocurre cruce, los hijos son copias exactas de los padres.

Mutación binaria. La función `mutar()` invierte cada gen (de 0 a 1 o de 1 a 0) de forma independiente con probabilidad $p_m = 0,05$.

Ejemplo manual completo. Con semilla `random.seed(2)` y la población inicial generada por `punto3_manual.py`, se aplicó el proceso completo de selección, cruce y mutación:

Cuadro 2: Ejemplo manual de selección, cruce y mutación (Punto 3)

Paso	Cromosoma	Costo / Beneficio / Aptitud
Padre 1 (torneo, $k = 3$)	0010000101	34 / 67 / 67.0
Padre 2 (torneo, $k = 3$)	0011110001	55 / 107 / 82.0
Punto de cruce: posición 4		
Hijo 1 (antes de mutar)	0010110001	47 / 92 / 92.0
Hijo 2 (antes de mutar)	0011000101	42 / 82 / 82.0
Hijo 1 (después de mutar, $p_m = 0,05$) Genes mutados (índices 0-9): 2, 3	0001110001	44 / 84 / 84.0
Hijo 2 (después de mutar, $p_m = 0,05$) Genes mutados (índices 0-9): ninguno	0011000101	42 / 82 / 82.0

Se observa que el Hijo 1 se obtuvo al combinar los primeros cuatro genes del Padre 1 (0010) con los seis genes finales del Padre 2 (110001), y el Hijo 2 de forma complementaria. La mutación modificó dos genes del Hijo 1 (posiciones 2 y 3), lo que redujo su beneficio de 92 a 84, mientras que el Hijo 2 no sufrió ningún cambio.

4 Punto 4 – Algoritmo genético completo

La función `ejecutar_algoritmo_genetico()` integra los componentes anteriores: genera la población inicial, evalúa la aptitud, selecciona padres por torneo, aplica cruce y mutación, conserva el mejor individuo mediante elitismo y repite el proceso hasta completar el número de generaciones definido como criterio de parada.

Cuadro 3: Configuración inicial del algoritmo genético

Componente	Diseño adoptado
Representación	Cromosoma binario de 10 genes
Tamaño de población	20 (configuración inicial)
Función de aptitud	Beneficio si $C(X) \leq 50$; penalización lineal $B(X) - \lambda(C(X) - 50)$ en caso contrario, con $\lambda = 5$
Selección	Torneo de tamaño 3
Cruce	Un punto, probabilidad 0.80
Mutación	Bit-flip gen a gen, probabilidad 0.05
Elitismo	Se conserva 1 individuo entre generaciones
Criterio de parada	Número fijo de generaciones (100 en la configuración inicial)

La penalización lineal permite que soluciones ligeramente inválidas sigan participando en la selección sin dominar a las soluciones factibles, evitando así descartar prematuramente material genético útil. Un valor de λ demasiado pequeño deja de penalizar efectivamente el exceso de presupuesto, favoreciendo individuos inválidos; un valor demasiado grande equivale a descartar de forma abrupta cualquier solución que exceda el presupuesto, reduciendo la diversidad disponible para la recombinación.

4.1 Punto 4 – Resultados de la ejecución completa

Se ejecutó el algoritmo cinco veces con la configuración inicial descrita en la tabla anterior, para evidenciar su naturaleza estocástica:

Cuadro 4: Cinco ejecuciones con la configuración inicial

Ejec.	Mejor benef.	Costo	Proyectos	Gen. del mejor	Tiempo (s)	Apt. prom. final
1	97	50	P1, P3, P4, P5, P8	1	0.00948	90.85
2	99	50	P1, P3, P4, P6, P9	6	0.00904	89.70
3	99	50	P1, P2, P3, P6, P7	10	0.00903	89.70
4	97	50	P1, P4, P6, P7, P8	0	0.00984	87.05
5	99	50	P1, P2, P3, P6, P7	11	0.00982	88.80

5 Punto 5 – Experimentación y análisis

5.1 Experimentos A, B y C

Cuadro 5: Resultados de los experimentos A, B y C

Exp.	Pob.	Gen.	Mut.	Mejor benef.	Costo	Proyectos	Gen. del mejor	Tiempo (s)
A	10	50	0.01	93	49	P1, P5, P8, P9, P10	43	0.00255
B	20	100	0.05	100	50	P1, P3, P6, P10	2	0.01008
C	50	200	0.10	100	50	P1, P3, P6, P10	1	0.05027

5.2 Gráficas de evolución

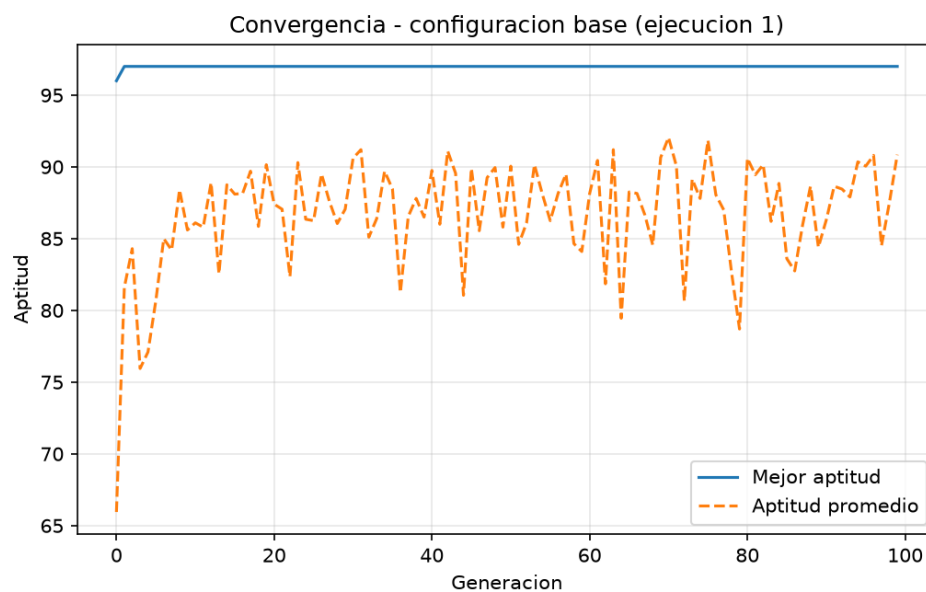


Figura 1: Convergencia de la configuración base

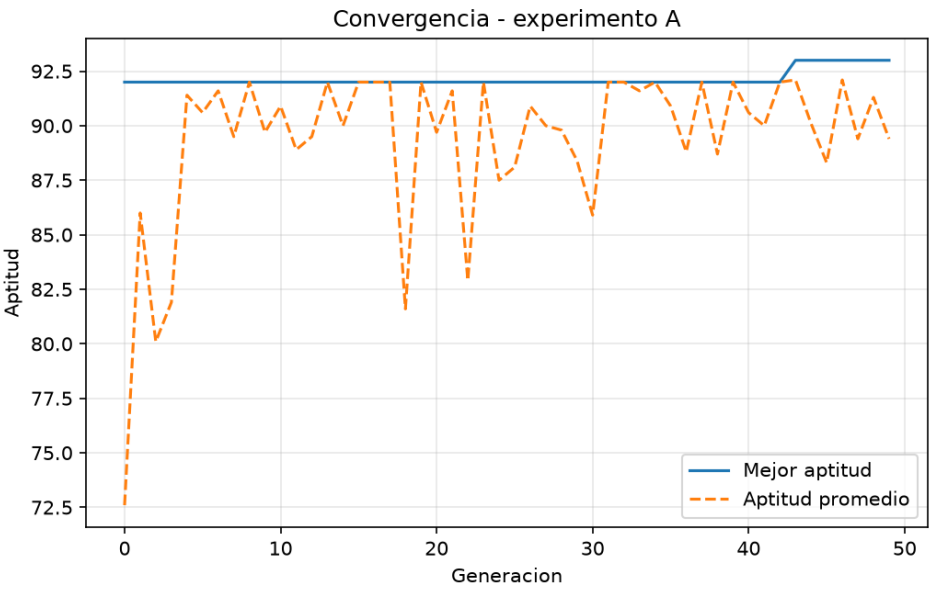


Figura 2: Convergencia del Experimento A

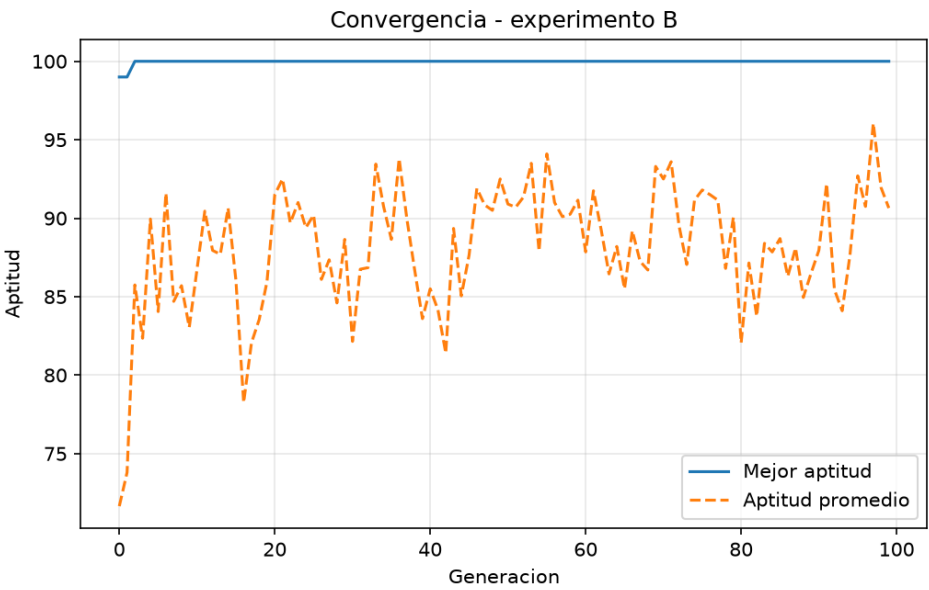


Figura 3: Convergencia del Experimento B

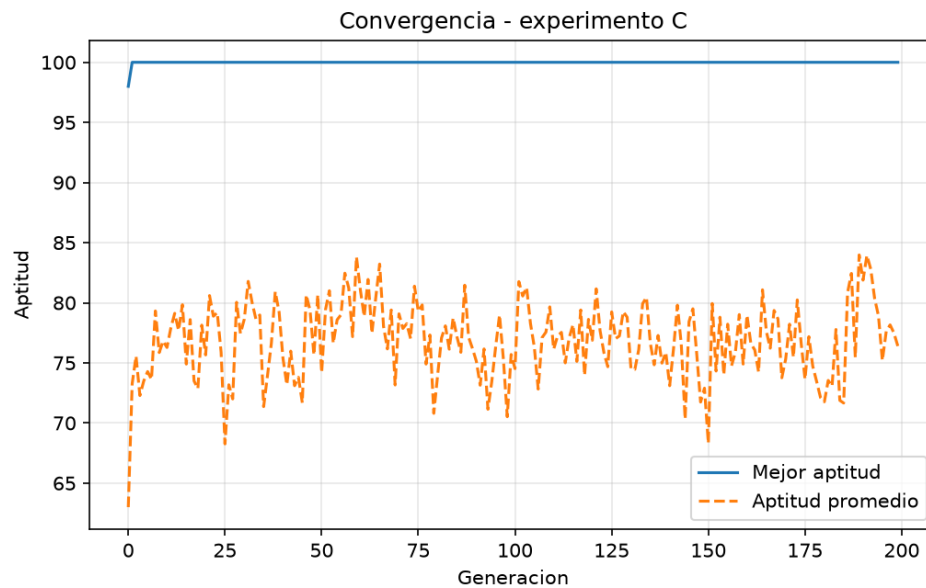


Figura 4: Convergencia del Experimento C

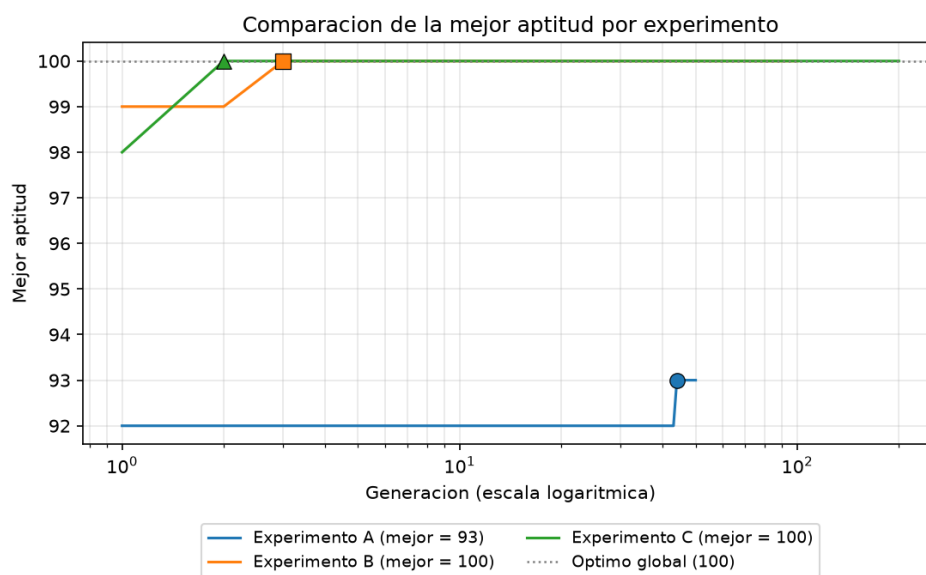


Figura 5: Comparación de la mejor aptitud entre experimentos

5.3 Explicación de la mejor solución encontrada

La mejor solución encontrada en el conjunto de ejecuciones corresponde al cromosoma $X = (1, 0, 1, 0, 0, 1, 0, 0, 0, 1)$, que selecciona los proyectos **P1**, **P3**, **P6** y **P10**, con un costo total de 50 unidades monetarias (exactamente el límite del presupuesto) y un beneficio total de 100 unidades. Esta solución fue encontrada por los experimentos B y C, y se verificó mediante búsqueda exhaustiva sobre las 1024 combinaciones posibles que corresponde al óptimo global del problema. Ninguna otra combinación factible produce un beneficio mayor.

5.4 Conclusiones sobre la influencia de los parámetros

a) ¿Qué efecto tuvo el tamaño de la población? Al aumentar la población de 10 (experimento A) a 20 y 50 individuos (experimentos B y C), el algoritmo pasó de estancarse en un óptimo local (beneficio 93) a alcanzar el óptimo global (beneficio 100) en muy pocas generaciones. Una población reducida limita la diversidad genética inicial y restringe el número de esquemas explorados simultáneamente.

b) ¿Cómo afectó la tasa de mutación a la diversidad? Una tasa de mutación baja (0.01), combinada con una población pequeña, resultó insuficiente para reintroducir diversidad una vez que la población convergió hacia un conjunto reducido de soluciones. Una tasa más alta (0.10) sostenida por una población grande mantuvo suficiente variabilidad para seguir explorando sin degradar la convergencia.

c) ¿Una tasa de mutación alta siempre produjo mejores resultados? No de forma aislada. En los experimentos realizados, la tasa de mutación y el tamaño de población variaron simultáneamente, por lo que su efecto no puede separarse por completo. Los resultados sugieren que una mutación alta solo es efectiva si la población es lo suficientemente grande para sostener la diversidad que introduce.

d) ¿El mejor individuo apareció antes de la última generación? Sí, en todos los casos. En las cinco ejecuciones de la configuración inicial, el mejor individuo apareció entre las generaciones 0 y 11; en los experimentos B y C apareció en las generaciones 2 y 1, respectivamente. Esto es consistente con el tamaño reducido del espacio de búsqueda del problema.

e) ¿El algoritmo obtuvo siempre la misma solución? No. Las cinco ejecuciones de la configuración inicial obtuvieron beneficios distintos (97, 99, 99, 97 y 99) y cromosomas distintos entre sí, lo que evidencia la naturaleza estocástica del algoritmo. Solo los experimentos con mayor población convergieron de manera consistente al óptimo global.

f) ¿Qué evidencia indicaría una convergencia prematura? En la gráfica de convergencia de la configuración base, la mejor aptitud se estabiliza desde la primera generación y permanece constante durante el resto de la ejecución, mientras que la aptitud promedio de la población continúa oscilando sin alcanzar ese valor. Esta brecha sostenida entre el mejor individuo y el promedio poblacional es el indicio característico de una convergencia prematura.

g) ¿Qué configuración recomendaría y por qué? Se recomienda una población de 20 a 50 individuos con una tasa de mutación entre 0.05 y 0.10, manteniendo el elitismo. Esta combinación mostró suficiente diversidad para escapar de óptimos locales sin sacrificar velocidad de convergencia, dado el tamaño reducido del espacio de búsqueda de este problema en particular.